

RITTMAN ANALYTICS · INTERNAL REFERENCE

Coding & Development Standards.

The single point of reference for how we build at Rittman Analytics. Tells you what to install, how we structure code, how we test it, how we ship it, and how we coordinate as a team. Read end-to-end on day one; thereafter, dip into the section that matches the work in front of you.

VERSION 1.0 12 May 2026	OWNER Head of Delivery Mark Rittman	AUDIENCE Engineers Permanent & contractor	CADENCE 6-monthly Or on trigger event
--------------------------------------	--	--	--

OUR VALUES

Everything in this document is downstream of three values that we hold ourselves and each other to at Rittman Analytics. When a convention seems inconvenient on a particular engagement, return to these and make the call.

VALUE 01

Pursue Expertise

A commitment to deep technical and individual expertise through continuous learning, ensuring our services and our people are industry-leading.

VALUE 02

Delivering on our Promise

A steadfast commitment to delivering on the promise we make to clients, ensuring solutions are delivered impeccably and add significant value.

VALUE 03

Integrity & Transparency

Transparent communication with our team and our clients, so that clients can rely on both the data and the commitment of the firm.

DOCUMENT CONTENTS

1	Purpose & how to use	p. 2	10	Releases, branches & PRs	p. 19
2	Delivery philosophy	p. 2	11	Ways of working	p. 20
3	Onboarding & environment	p. 3	12	Jira	p. 21
4	The Wire Framework	p. 4	13	Confluence	p. 21
5	dbt standards	p. 5	14	GitHub	p. 21
6	dbt migration · Snowflake → BigQuery	p. 9	15	Harvest & Harvest Forecast	p. 22
7	Other technology standards	p. 13	16	Line management & escalation	p. 22
8	Testing & QA	p. 17	17	Final notes & maintenance	p. 23
9	Project planning & engagement models	p. 18	A	Appendix · Reference links	p. 23

SECTION 1 Purpose & how to use this document

This is the single point of reference for *how* we build at Rittman Analytics. It tells you what to install, how we structure code, how we test it, how we ship it, and how we coordinate as a team. You should be able to read this document, complete the onboarding actions in Section 3, and start delivering on a client engagement without further hand-holding. Where deeper detail exists in a Confluence page, this guide links out rather than duplicating.

READ ORDER FOR NEW JOINERS

- 1 Section 2 (Delivery philosophy) and Section 3 (Onboarding) on day one.
- 2 Section 4 (Wire Framework) and Section 5 (technology standards for the tools on your engagement) before your first piece of work.
- 3 Section 11 (Ways of working) and Section 16 (Line management) once you're on a project team.
- 4 Everything else as a reference.

SECTION 2 Delivery philosophy

Three principles run through everything below. If you're ever unsure how to make a decision on a project, fall back to these.



START WITH DESIGN, NOT CODE

We always build a conceptual model and physical model — with sign-off from the business — before writing transformation code. The cost of changing a model on a whiteboard is minutes. The cost of changing it after dbt and LookML are written is days.



THE WAREHOUSE IS THE SOURCE OF TRUTH

Business logic — calculations, definitions of metrics, conformed dimensions — lives in dbt, never in Looker. Looker is a thin semantic and presentation layer. This is non-negotiable. The reasoning is in the *RA Development & Design Guide for dbt & Looker* — read it on day one.



AI-AUGMENTED DELIVERY IS THE DEFAULT, NOT THE EXCEPTION

Our Wire Framework, built on Claude Code and MCP servers, is the standard way we deliver projects. Manual development is a fallback for situations where the framework can't be applied — greenfield experimentation, tiny one-off changes, environments where Claude Code can't run. If you find yourself coding manually on a normal client engagement, that needs a conversation with your Head of Delivery first. It's an exception, not the norm.

RULE OF THUMB

Standards aren't red tape. Design before code, test before merge, escalate early, document the deviation. These are the mechanism by which our three values become operationally real on a project. Skipping them isn't pragmatism; it's drifting from what we say we are.

SECTION 3 Onboarding & environment setup

Complete these on day one. If you're a contractor and any aren't provisioned within 48 hours of starting, raise it with your engagement lead.

3.1 ACCOUNTS & ACCESS

SYSTEM	PURPOSE	PROVISIONED BY	DONE
Google Workspace	Email, Docs, Sheets, Slides, Drive, Calendar (@rittmananalytics.com)	Operations	☐
Slack	Day-to-day communication	Operations	☐
Jira	Ticket tracking	Head of Delivery	☐
Confluence	Documentation	Head of Delivery	☐
GitHub	Source control for all RA repos (rittmananalytics.org)	Head of Delivery	☐
Harvest	Time and project tracking	Operations	☐
Harvest Forecast	Resource scheduling	Head of Delivery	☐
Fathom	Meeting recording & transcripts	Operations	☐
Claude (Team)	AI development partner — required for Wire	Operations	☐
GCP	BigQuery, Looker, Cloud Run — project-specific	Engagement lead	☐
dbt Cloud	Per-client, where used	Engagement lead	☐
Fivetran	Per-client, where used	Engagement lead	☐

3.2 LOCAL DEVELOPMENT ENVIRONMENT

SUPPORTED PLATFORM

macOS only, on the latest released version. All RA-issued laptops are Macs; all internal tooling, install scripts, and onboarding assume macOS. Contractors are welcome to use Windows or Linux, but on the understanding that any platform-specific support issues — install failures, path differences, `gccloud` quirks, Claude Code CLI differences — are theirs to resolve. We won't refuse to help, but it isn't something we maintain documentation or test coverage for.

Required

- Git and GitHub CLI (`gh`) or GitHub Desktop.
- VS Code (1.98+) with the Claude Code extension, or the Claude Code CLI (`claude`). Wire will not run without this.
- Python 3.11+ with a virtual-environment workflow (`venv` , `uv` or `pyenv` — pick one and stick with it).
- Google Cloud CLI (`gcloud`) authenticated against your @rittmananalytics.com identity.
- dbt Core (`pip install dbt-bigquery` or `dbt-snowflake`) — even where the client uses dbt Cloud, useful for local development.
- sqlfluff (`pip install sqlfluff`) for automated SQL linting.

Recommended VS Code extensions

- dbt Power User
- Better Jinja
- Python
- GitLens
- vscode-mermaid-preview
- sqlfluff

3.3 First-day Confluence reading

- 1 This document.
- 2 *Wire Framework Consultant Handbook* — the operating manual.
- 3 *RA Development & Design Guide for dbt & Looker* — the design philosophy.
- 4 *RA Standard Operating Procedure Directory* — the index into deeper material.

SECTION 4 The Wire Framework — our preferred method of delivery

Wire is Rittman Analytics' proprietary AI-accelerated delivery methodology. It is the default way we deliver every client engagement. It encodes the standards in this document as executable workflow specifications that Claude Code runs against, so artifacts (dbt models, LookML, tests, runbooks, training) are generated *to* the standards rather than checked against them after the fact.

4.1 WHY IT'S THE DEFAULT

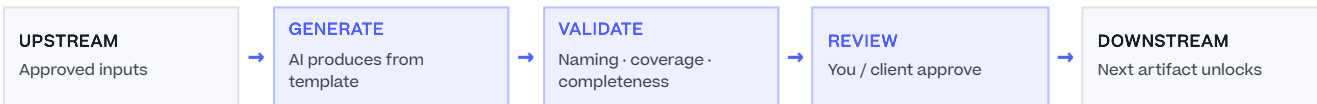
- **Naming, structure and tests are guaranteed consistent.** Templates are baked into the workflow specs — a model generated on a Tuesday morning by one consultant looks identical to one generated by another on a different project.
- **Phase discipline is automatic.** You can't generate dbt code until the data model is approved. You can't deploy until UAT is signed off. The framework enforces this through prerequisite checks.
- **Audit trail is free.** Every artifact has a `generate → validate → review` lifecycle recorded in `status.md`. Clients and the Head of Delivery can see exactly where a project is at any moment.
- **Speed.** A standard dbt model layer that would take a senior consultant a week of manual development now takes a day to generate, validate and review.

4.2 THE FIVE PROJECT TYPES

TYPE	WHEN TO USE	DURATION
full_platform	New end-to-end build: SOW → production dashboards + trained users	2–3 weeks
pipeline_dbt	New data pipeline + dbt transformation; client already has BI	1–2 weeks
dbt_development	Data is already in the warehouse; building transformation layer	1 week
dashboard_extension	New dashboards on an existing semantic layer	3–5 days
enablement	Training and documentation only	2–3 days

4.3 THE ARTIFACT LIFECYCLE

Every artifact passes three gates. An artifact does not progress, and downstream artifacts will refuse to generate, until all three gates pass.



4.4 WHEN MANUAL DEVELOPMENT IS APPROPRIATE (THE FALLBACK)

Manual coding is acceptable, with Head of Delivery approval, when: the engagement is a tiny, one-off change where framework overhead exceeds the work itself; you're prototyping something experimental that won't ship to production; the technology stack is outside the framework's current coverage (raise it — the framework is meant to grow); or Claude Code cannot run in the client's environment. **The standards in this document still apply.** The framework encodes them; it doesn't replace them.

4.5 INSTALLATION

Four commands inside Claude Code. Full walkthrough in the *Wire Framework Developer Onboarding deck*.

```
$ /plugin marketplace add rittmananalytics/wire-plugin
$ /plugin install wire@rittman-analytics
$ /reload-plugins
$ /wire:help
```

If `/wire:help` returns the command tree (discovery, requirements, pipeline, dbt, semantic, dashboards, release, meta), you're done. Run `/wire:new` to spin up your first engagement.

SECTION 5 dbt standards

These are the current canonical conventions.

5.1 PROJECT STRUCTURE

Three layers: `staging`, `integration`, `warehouse`. Each layer is organised into **entity groups** (`core`, plus one per business domain), and models follow lineage strictly — staging selects from sources, integration selects from staging, warehouse selects from integration. A warehouse model *may* select directly from staging where no integration step is necessary; integration models exist primarily to enrich data by joining multiple sources.

```
models/
├── field_descriptions.md
├── warehouse/
│   ├── wh_core/
│   │   ├── wh_core__user_dim.sql
│   │   ├── wh_core__country_dim.sql
│   │   └── wh_<entity_group>/
│   │       ├── wh_<entity_group>__<entity>.dim.sql
│   │       ├── wh_<entity_group>__<entity>.fact.sql
│   │       └── wh_<entity_group>__<entity>.xa.sql
│   └── integration/
│       ├── int_core/
│       │   ├── int_core__users.sql
│       │   ├── int_core__geographies.sql
│       │   └── int_<entity_group>/
│       │       └── int_<entity_group>__<entity>.sql
│       └── staging/
│           ├── stg_core/
│           │   ├── _sources.yml
│           │   ├── stg_core__users.sql
│           │   ├── stg_core__geographies.sql
│           │   └── stg_<entity>/
│           │       ├── _sources.yml
│           │       └── stg_<entity>__<table>.sql
```

5.2 MODEL NAMING

LAYER	PATTERN	EXAMPLE
Staging	<code>stg_<entity_group>__<table>.sql</code>	<code>stg_core__users.sql</code>
Integration	<code>int_<entity_group>__<entity>.sql</code>	<code>int_core__users.sql</code>
Warehouse dimension	<code>wh_<entity_group>__<entity>.dim.sql</code>	<code>wh_core__user_dim.sql</code>
Warehouse fact	<code>wh_<entity_group>__<entity>.fact.sql</code>	<code>wh_sales__order_fact.sql</code>
Warehouse XA	<code>wh_<entity_group>__<entity>.xa.sql</code>	<code>wh_sales__order_d_xa.sql</code>

The suffix is `_fact` (full word), not `_fct`. The `wh_` / `int_` / `stg_` layer prefix and the entity-group separator (`__`, double underscore) are mandatory. Object names follow the source where natural — plural sources stay plural (`users`, `geographies`) at the staging and integration layers; warehouse dimensions and facts use the singular business entity name (`user_dim`, `order_fact`).

5.3 FIELD NAMING

TYPE	PATTERN	EXAMPLE
Natural key	<descriptive_name>_natural_key	subscription_natural_key
Primary key	<entity>_pk	subscription_pk
Foreign key	<entity>_fk	subscription_fk
Timestamp (UTC)	<event>_ts	created_ts
Timestamp (non-UTC)	<event>_<tz>_ts	created_cet_ts
Boolean	is_... / has_... / was_...	is_active, has_subscription
Revenue	<entity>_<purpose>_amount	user_account_balance_amount
Common entity fields	<entity>_<field>	customer_name

- All `snake_case`, lowercase, in business terminology rather than source terminology.
- Avoid SQL reserved words as column names.
- **Primary and foreign key columns must be generated using** `dbt_utils.generate_surrogate_key` — never hand-rolled, never looked up via a separate query.
- **Prices** must be converted to decimal currency; integer-cents columns are not stored as the canonical price.
- **All type casts go through dbt macros** — `dbt.type_string()`, `dbt.type_numeric()`, `dbt.type_boolean()`, `dbt.type_timestamp()`, `type_date()`. Don't hand-write `cast(x as varchar)` — the macros abstract dialect differences and make migrations less painful.

5.4 FIELD ORDERING INSIDE A SELECT

Use `{# keys #}`, `{# attributes #}`, `{# metrics #}`, `{# booleans #}`, `{# temporal data types #}` comment markers in staging models to make the grouping visually obvious. Order:

- Keys** — natural keys, primary key, foreign keys
- Attributes** — dimensional fields, descriptors
- Indexes / ranks** — ordering or ranking columns
- Metrics** — numeric measures, amounts
- Booleans** — `is_`, `has_`, `was_` flags
- Temporal data types** — dates & timestamps last

5.5 SQL STYLE

RULE	DETAIL
Indentation	4 spaces (not tabs). Predicates line up with the <code>where</code> keyword.
Line length	Max 80 characters.
Case	Lowercase field names and function names.
Aliases	Always use the <code>as</code> keyword.
Joins	Explicit: <code>inner join</code> , <code>left join</code> , etc. — never bare <code>join</code> . <code>right join</code> usually means you should swap which table is in the <code>from</code> .
Table aliases	Full descriptive names (<code>customer</code>), not initialisms (<code>c</code>).
Column prefixes	When joining two or more tables, always prefix column names with the table alias. Single-table selects don't need prefixes.
Aggregations	Execute as early as possible — aggregate before joining wherever feasible.
group by / order by	Use column names, not numeric positions.
Union	Prefer <code>union all</code> to <code>union distinct</code> for performance.
CTEs from refs	Prefix with <code>s_</code> (e.g. <code>s_users</code>).
Final CTE	Always named <code>final</code> ; the model ends with <code>select * from final</code> .
Comments	Explain <i>why</i> , not <i>what</i> . Required for non-obvious CTEs, complex joins, coalesce, case when.

RULE OF THUMB

Do not optimise for a smaller number of lines of code. Newlines are cheap; brain time is expensive.

5.6 CTES

- All `{{ ref('...') }}` and `{{ source('...') }}` statements live in CTEs at the top of the file, with the CTE name prefixed `s_`.
- Where performance permits, one CTE = one logical unit of work.
- CTE names are as verbose as needed to convey what they do.
- CTEs containing notable or confusing logic carry a comment above them.
- The final CTE is named `final`. This makes it easy to debug a model by temporarily changing the last line to `select * from <some_earlier_cte>` without commenting out code.

5.7 CANONICAL STAGING MODEL

The structural template for any staging model. Note the `{# section #}` markers for the field-ordering groups, the `lower(cast(...))` pattern for string IDs, and `dbt.type_*` macros for every cast.

```
{{
  config(
    description = 'A description of the model to help developers'
  )
}}

with s_users as (

  select * from {{ source('back_office', 'user_accounts') }}

),

rename_and_cast as (

  select

    {# keys #}
    lower(cast(id as {{ dbt.type_string() }} ))      as user_natural_key,
    {# attributes #}
    lower(cast(name as {{ dbt.type_string() }} ))    as user_name,
    {# metrics #}
    cast(account_balance as {{ dbt.type_numeric() }} ) as user_account_balance_amount,
    {# booleans #}
    cast(status as {{ dbt.type_boolean() }} )        as user_status,
    {# temporal data types #}
    cast(created_date as {{ type_date() }} )          as user_created_dt,
    cast(update_at as {{ dbt.type_timestamp() }} )   as user_update_ts

  from s_users

),

final as (

  select * from rename_and_cast

)

select * from final
```

For integration and warehouse models, keep the same `s_<ref>` → working-CTE → `final` structure. The `rename_and_cast` step is only relevant at the staging layer because that's where source columns first get business-friendly names and types.

5.8 MATERIALISATION

LAYER	DEFAULT MATERIALISATION
Staging	View or ephemeral / CTE — keep lightweight
Integration	View or ephemeral / CTE — table only if performance requires it
Warehouse	Always table; consider partition and cluster keys

Layer-wide configuration goes in `dbt_project.yml`. Model-specific configuration (unique keys, partitioning, descriptions) goes in the file's `{{ config() }}` block.

5.9 YAML STYLE

- 2-space indentation.
- List items indented.
- New line between list items that are themselves dictionaries.
- 80-character line limit.

5.10 JINJA STYLE

- Spaces on the inside of delimiters: `{{ this }}`, not `{{this}}`.
- Strike a balance between readable Jinja and readable SQL — when in doubt, lean toward readable SQL and accept more Jinja verbosity.
- Use newlines to visually demarcate logical Jinja blocks.

5.11 DOCUMENTATION

- **Warehouse-layer columns must be documented.** No exceptions.
- Use dbt doc blocks for any field description that would otherwise be duplicated across models — store all descriptions in `models/field_descriptions.md` and reference them via `"{{ doc('field_name') }}"`.
- Documentation coverage can be enforced via `dbt-meta-testing`; enable in CI on engagements where coverage gaps are a recurring problem.

5.12 SQLFLUFF

If a `.sqlfluff` config file is present in the project root, run `sqlfluff lint models/ --dialect <bigquery|snowflake|postgres>` before raising a PR. CI will run it too — getting it green locally first is faster than going round the loop with the bot.

5.13 COMMON VIOLATIONS

✗ Don't

- Skip the entity-group folder structure (no flat `warehouse/` with all dims at the top level).
- Use `_fct` (the suffix is `_fact`).
- Put `{{ ref() }}` or `{{ source() }}` calls outside top CTEs.
- Use implicit joins or bare `join`.
- Use single-letter or initialism aliases (`c` for customer).
- Hand-write `cast(x as string)` — go through the `dbt.type_*` macros.
- Hand-roll primary or foreign keys — use `dbt_utils.generate_surrogate_key`.
- Leave warehouse columns undocumented.
- Select from sources in non-staging models.
- Use `union distinct` without good reason.
- Mix tabs and spaces.

✓ Do

- Use the layered `stg_` → `int_` → `wh_` naming with entity-group prefixes.
- All refs in top CTEs, prefixed `s_`.
- Explicit join types.
- Full descriptive table aliases.
- 4-space indentation; predicates aligned to `where`.
- Casts through `dbt.type_*` macros.
- Keys via `dbt_utils.generate_surrogate_key`.
- Test every primary key (`unique` + `not null`).
- Document warehouse columns 100%.
- Respect layer boundaries; aggregate before joining where feasible.

SECTION 6 dbt migration — Snowflake → BigQuery

Snowflake-to-BigQuery migration is one example of a platform migration. Treat it as a project type with three distinct phases and a defined exit gate at each.

6.0 PREFERRED APPROACH WHEN THE MIGRATION ALSO INVOLVES LOOKER

Where the engagement is migrating *both* the data platform *and* the BI tool (typically Snowflake + Tableau → BigQuery + Looker), our strong preference is to **implement the Looker semantic layer on the platform being migrated *from*, get it working there alongside the existing BI tool, and only then cut over to the target platform.**

- Stand up Looker against the source platform** (e.g. Snowflake). Where the existing dbt transformations are sufficient, point Looker straight at them. Where they aren't, build the additional dbt transformations needed on the source platform first.
- Build the LookML semantic layer and reports on the source platform.** Validate measures, dimensions and explores against the existing BI tool's output until business users are satisfied that Looker reproduces what they had.
- Get business sign-off on the Looker reports against the source platform.** Users have agreed Looker, against their existing data, gives them what they need.
- Then, and only then, cut over the underlying platform.** The dbt project moves to BigQuery using the migration phases below; the LookML repo is reprinted at the new BigQuery connection; reports are validated like-for-like between the old and new platforms.

WHY THIS WORKS

The BI migration (which is the bit business users actually see) is decoupled from the platform migration (which is the bit they don't care about until it goes wrong). Users adopt Looker against familiar, trusted data. Once they trust Looker, the platform swap is invisible — same dashboards, same numbers, same explore navigation.

The like-for-like Looker validation step at cut-over is mandatory: every published report is run on both platforms, row counts and aggregate values compared, and any discrepancy investigated before sign-off (see Section 6.5). Where the source platform is being decommissioned on a fixed date, or Looker can't connect to it, do the platform migration first — but flag the additional user-adoption risk explicitly to the client and to your Head of Delivery.

6.1 PHASE 1 — ASSESSMENT (BEFORE ANY MIGRATION WORK)

Produce a written assessment containing:

- Total model inventory by layer (staging / integration / warehouse / marts).
- Custom macro inventory.
- Platform-specific code catalogue — every Snowflake-specific function or syntax that will need translation (Section 6.3 is the lookup).
- Package audit — every `packages.yml` entry checked for BigQuery compatibility, with target versions identified.
- Pre-migration unit-test baseline — created on the Snowflake side and recorded as "N tests, all passing" before anything is migrated.

The pre-migration baseline is non-negotiable. Without it you have no objective measure of whether the migration changed behaviour.

6.2 PHASE 2 — EXECUTION

Execute in this strict order, clearing `target/` and recompiling between each step. Track the error count down to zero, category by category:

- | | |
|---|---|
| <ol style="list-style-type: none"> Environment setup — new target in <code>profiles.yml</code>,
<code>dbt debug --target bigquery</code> passes, target dataset exists,
<code>dbt deps</code> succeeds with updated packages. Initial compilation — <code>dbt compile --target bigquery</code>; catalogue every error, classify by category. Data type fixes Date / time function fixes — use <code>dbt_utils</code> dispatch macros where they exist | <ol style="list-style-type: none"> String function fixes NULL-handling fixes Identifier quoting fixes Array / struct / JSON fixes Custom macro fixes — rewrite or create dispatch macros Package-specific & remaining — finish at zero compilation errors |
|---|---|

Every fix is recorded in `migration_changes.md` with model name, line, before/after snippet, and rationale. This is the audit trail clients will ask for.

6.3 SQL DIALECT LOOKUP — SNOWFLAKE → BIGQUERY

The most frequent translations. The full reference (also including Databricks) sits alongside the Wire Framework migration workflow spec.

Date arithmetic and truncation

OPERATION	SNOWFLAKE	BIGQUERY
Add days	<code>DATEADD(day, 7, d)</code>	<code>DATE_ADD(d, INTERVAL 7 DAY)</code>
Subtract days	<code>DATEADD(day, -7, d)</code>	<code>DATE_SUB(d, INTERVAL 7 DAY)</code>
Add months	<code>DATEADD(month, 3, d)</code>	<code>DATE_ADD(d, INTERVAL 3 MONTH)</code>
Date diff (days)	<code>DATEDIFF(day, start, end)</code>	<code>DATE_DIFF(end, start, DAY)</code> ⚠ END FIRST
Truncate to month	<code>DATE_TRUNC('month', d)</code>	<code>DATE_TRUNC(d, MONTH)</code> ⚠ BARE KEYWORD
Last day of month	<code>LAST_DAY(d)</code>	<code>LAST_DAY(d, MONTH)</code>

ARGUMENT-ORDER TRAP

`DATEDIFF` flips between dialects. Snowflake takes `(part, start, end)`; BigQuery takes `(end, start, part)`. This is the single most common silent-error bug in a migration — the SQL compiles fine and returns the wrong sign.

Date extraction

OPERATION	SNOWFLAKE	BIGQUERY
Day of week	<code>DAYOFWEEK(d)</code>	<code>EXTRACT(DAYOFWEEK FROM d)</code>
Week number	<code>WEEKOFYEAR(d)</code>	<code>EXTRACT(WEEK FROM d)</code>

⚠ Day-of-week numbering differs between Snowflake (depends on `WEEK_START` parameter, defaults to Monday=0) and BigQuery (Sunday=1, Saturday=7). Always validate downstream logic that depends on weekday numbers.

Date formatting and parsing

OPERATION	SNOWFLAKE	BIGQUERY
Format date	<code>TO_CHAR(d, 'YYYY-MM-DD')</code>	<code>FORMAT_DATE('%Y-%m-%d', d)</code>
Parse date	<code>TO_DATE(s, 'YYYY-MM-DD')</code>	<code>PARSE_DATE('%Y-%m-%d', s)</code>
Format timestamp	<code>TO_CHAR(ts, 'YYYY-MM-DD HH24:MI:SS')</code>	<code>FORMAT_TIMESTAMP('%Y-%m-%d %H:%M:%S', ts)</code>

Format-code differences: `YYYY → %Y`, `MM → %m`, `DD → %d`, `HH24 → %H`, `MI → %M`, `SS → %S`, `MON → %b`, `MMMM → %B`.

Strings

OPERATION	SNOWFLAKE	BIGQUERY
Concat with separator	<code>CONCAT_WS(',', a, b, c)</code>	<code>ARRAY_TO_STRING([a,b,c], ',')</code>
Find position	<code>POSITION(sub IN s)</code>	<code>STRPOS(s, sub)</code>
Regex extract	<code>REGEXP_SUBSTR(s, r)</code>	<code>REGEXP_EXTRACT(s, r'...')</code> ⚠ RAW-STRING
Regex match	<code>REGEXP_LIKE(s, r)</code>	<code>REGEXP_CONTAINS(s, r)</code>
Split and get part	<code>SPLIT_PART(s, d, n+1)</code>	<code>SPLIT(s, d)[OFFSET(n)]</code> ⚠ 0-INDEXED
SHA256 hash	<code>SHA2(s, 256)</code>	<code>SHA256(s)</code>

6.3 SQL DIALECT LOOKUP (CONTINUED)

Casting

OPERATION	SNOWFLAKE	BIGQUERY
Safe cast	<code>TRY_CAST(x AS INTEGER)</code>	<code>SAFE_CAST(x AS INT64)</code>
To integer	<code>CAST(x AS INTEGER)</code>	<code>CAST(x AS INT64)</code>
To string	<code>CAST(x AS VARCHAR)</code>	<code>CAST(x AS STRING)</code>
To float	<code>CAST(x AS FLOAT)</code>	<code>CAST(x AS FLOAT64)</code>
To decimal	<code>CAST(x AS NUMBER(38,2))</code>	<code>CAST(x AS NUMERIC)</code>

Aggregates and conditionals

OPERATION	SNOWFLAKE	BIGQUERY
Conditional count	<code>COUNT_IF(condition)</code>	<code>COUNTIF(condition)</code>
If / else	<code>IFF(cond, then, else)</code>	<code>IF(cond, then, else)</code>
If null	<code>NVL(a, default)</code>	<code>IFNULL(a, default)</code>
Safe divide	<code>DIV0NULL(a, b)</code>	<code>SAFE_DIVIDE(a, b)</code>
Median	<code>MEDIAN(x)</code>	<code>PERCENTILE_CONT(x, 0.5) OVER()</code>
String agg	<code>LISTAGG(x, ',')</code>	<code>STRING_AGG(x, ',')</code>

JSON and arrays

Snowflake's `VARIANT` path notation (`j:field`, `j:a.b`) has no direct BigQuery equivalent; rewrite to `JSON_VALUE(j, '$.field')` / `JSON_VALUE(j, '$.a.b')`. Snowflake's `LATERAL FLATTEN(input => arr)` rewrites to BigQuery's `UNNEST(arr) AS elem`.

Hashing for surrogate keys

Snowflake's `HASH(a, b)` becomes BigQuery's `FARM_FINGERPRINT(CONCAT(a, b))`.

VERIFY POST-MIGRATION

Any existing `_pk` columns: if `dbt_utils.surrogate_key()` is used (the standard), the underlying hash function differs between dialects, so primary-key values *will* change. Downstream FK joins still work because both sides regenerate from the same dialect, but any historical PK references in BI dashboards or external systems must be re-validated.

6.4 PHASE 3 – VALIDATION

Don't close a migration until all of this is green:

- `dbt build --target bigquery` succeeds with zero errors.
- All unit tests pass and match the pre-migration baseline count exactly.
- All data tests pass.
- **Row-count parity** documented for every key warehouse model — source row count vs target row count, in a table in the migration write-up.
- **Aggregate parity** documented for the top-N metrics — for each, source value vs target value, with difference and whether acceptable (rounding from `NUMERIC` precision is usually acceptable; anything larger needs investigation).
- No unexpected NULLs introduced.
- Timezone behaviour verified — Snowflake `TIMESTAMP_TZ` vs BigQuery `TIMESTAMP` (always UTC) is a frequent source of off-by-N-hours errors.
- Performance baseline captured on BigQuery for the most expensive models.
- Rollback plan documented and the old Snowflake target retained in `profiles.yml` for the rollback window agreed with the client.

6.5 MIGRATION CHECKLIST TEMPLATE

Copy into `migration_checklist.md` in the project repo and tick items as you go. Sign-off rows are filled by the migration lead, data engineer, analytics engineer, and client stakeholder.

Pre-Migration

- ☐ Migration assessment completed
- ☐ Pre-migration unit-test baseline created and passing
- ☐ Baseline documented: N tests, all passing
- ☐ Packages audited for BigQuery compatibility
- ☐ BigQuery credentials configured; `dbt debug` passes
- ☐ Jira epic and tasks created

Looker-First IF IN SCOPE

- ☐ Looker connection configured against the source platform
- ☐ Additional dbt transformations (if needed) built on source platform
- ☐ LookML semantic layer built on source platform
- ☐ Looker reports built on source platform
- ☐ Reports validated against incumbent BI tool
- ☐ Business sign-off on Looker reports against source platform

Execution

- ☐ Environment setup complete
- ☐ Initial compilation run; errors catalogued by category
- ☐ Data type fixes complete; error count reduced X → Y
- ☐ Date / time fixes complete; error count reduced X → Y
- ☐ String fixes complete; error count reduced X → Y
- ☐ NULL handling complete; error count reduced X → Y
- ☐ Identifier quoting complete; error count reduced X → Y
- ☐ Array / Struct / JSON complete; error count reduced X → Y
- ☐ Custom macros complete; error count reduced X → Y
- ☐ Package-specific complete
- ☐ 0 compilation errors achieved
- ☐ All changes documented in `migration_changes.md`

Validation

- ☐ `dbt build` succeeds with 0 errors
- ☐ Unit tests pass; count matches pre-migration baseline
- ☐ Data tests pass
- ☐ Row-count parity table complete
- ☐ Aggregate parity table complete
- ☐ No unexpected NULLs introduced
- ☐ Timezone behaviour verified

Looker Cut-Over Validation IF IN SCOPE

- ☐ LookML repo repointed at BigQuery connection
- ☐ All published Looker reports run on both platforms
- ☐ Like-for-like row counts compared per report; discrepancies investigated
- ☐ Like-for-like aggregate values compared per report; discrepancies investigated
- ☐ Performance comparison documented per report
- ☐ Final business sign-off on Looker reports against BigQuery

Post-Migration

- ☐ `migration_changes.md` finalised
- ☐ Rollback plan documented; old target retained
- ☐ CI/CD updated to new target
- ☐ Performance baseline captured
- ☐ Team and stakeholders notified

SIGN-OFF

Each section above carries a sign-off row in the actual checklist: migration lead, data engineer, analytics engineer, client stakeholder. No section is "done" until all four have signed.

SECTION 7 Other technology standards

7.1 FIVETRAN

- Connector setup documented in Confluence (client space) **before** the connector goes live: schema name, source-credentials owner, sync frequency, MAR expectations.
- BigQuery schema names follow `fivetran_<source_system>` (e.g. `fivetran_hubspot`, `fivetran_stripe`).
- Prefer CDC replication where supported; full re-sync only where CDC isn't available. Document the choice and the cost implication in the Wire `pipeline_design` artifact.
- Sensitive columns** (safeguarding-sensitive, GDPR-restricted, or free-text fields likely to contain PII) are excluded at the Fivetran level, not after the fact. A common pattern is to exclude free-text note/comment columns entirely from replication when the source system is used in a regulated or safeguarding context.
- Set `freshness` thresholds on every Fivetran-loaded `_sources.yml` entry: `warn_after` and `error_after` realistic for the connector's sync schedule (30/60 min for live API data, 120/1560 min for daily loads).
- Debugging:** if a Fivetran MCP server tool isn't exposing correctly, the current workaround is direct `curl` against the Fivetran REST API — document the issue in the client's Slack channel and raise an internal ticket.

7.2 LOOKER / LOOKML

Our Looker semantic layer uses a **layered refinement architecture** built on top of an auto-generated base layer. Each layer has a single defined purpose, and refinements (`+`) are the mechanism by which higher layers extend or override lower ones. The base layer is auto-generated: **droughty** is the established open-source option; the Wire Framework can also auto-generate LookML to the same conventions (via `/dp:semantic_layer:generate`). Either way, you should not hand-write base-layer files.

Project structure

```
analytics_warehouse
├── manifest.lkml // LookML manifest
├── analytics_warehouse
│   └── lookml
│       ├── aggregate/
│       │   └── agg_<entity_group>.layer.lkml // Refinements adding measures
│       ├── base/
│       │   ├── _aggregate.layer.lkml // Generated measures
│       │   └── _base.layer.lkml // Generated dimensions
│       ├── int/
│       │   └── int_explore_<entity_group>.explore.lkml // Explores
│       ├── model/
│       │   └── analytics_warehouse.model.lkml // Project model
│       └── staging/
│           └── stg_<entity_group>.layer.lkml // Refinements to dimensions
```

Layer purposes

LAYER	PURPOSE
base/	Auto-generated. Maps 1:1 to warehouse schema. Two files: <code>_base.layer.lkml</code> (dimensions) and <code>_aggregate.layer.lkml</code> (measures generated from warehouse columns). Field names and types match the warehouse exactly. No business logic, no user-facing labels — that's what higher layers are for. Do not hand-edit.
staging/	Refinements to dimensions only. One file per entity group: <code>stg_<entity_group>.layer.lkml</code> . Uses <code>+</code> to override or enhance base dimensions: user-friendly <code>label:</code> and <code>group_label:</code> , HTML formatting, value formatters, drill sets, parameters, toggles, hiding fields not needed by business users (PKs, FKs). Contains no calculated or aggregate measures.
aggregate/	Measures only. One file per entity group: <code>agg_<entity_group>.layer.lkml</code> . Refinements add or override measures: averages, medians, business-specific metrics, time-based comparisons, ratios, conversion rates. All aggregate metrics defined here. Measures that exist only in the semantic layer get the suffix <code>_calculated</code> .
int/	Explores and joins. One file per entity group, named <code>int_explore_<entity_group>.explore.lkml</code> . Uses <code>include:</code> to pull in base, staging, aggregate (in that order). Defines relationships, default fields, access filters, Quick Starts, Aggregate Awareness.
model/	One file, the project model. Defines the connection (one only), sets <code>case_sensitive: no</code> , and uses <code>include:</code> to pull in every <code>int_explore_<entity_group>.explore.lkml</code> .

7.2 LOOKER / LOOKML (CONTINUED)

Include order

The `include:` order in integration-layer files determines which refinements are applied. Base files come first, then staging, then aggregate:

```
include: "/analytics_warehouse/lookml/base/_base.layer.lkml"
include: "/analytics_warehouse/lookml/base/_aggregate.layer.lkml"
include: "/analytics_warehouse/lookml/staging/*"
include: "/analytics_warehouse/lookml/aggregate/*"
```

Refinement rules

- Always use refinements (`+`) to modify existing views. Refine a view exactly **once** in staging and **once** in aggregate — multiple refinements across files create duplicate definitions and increase risk of data-quality bugs.
- View names being refined use the warehouse table name as their identifier: `view: +wh_<entity_group>_<entity>`.
- Use the `_calculated` suffix on any dimension or measure that exists only in the semantic layer with no corresponding warehouse column.

Naming & file suffixes

- Files in **staging** and **aggregate** use the suffix `.layer.lkml` — widely adopted in the Looker community, what the Looker IDE expects.
- Files in **integration** use `.explore.lkml` so Looker recognises them as Explores.
- View names match warehouse table names. Explore names match business processes, not underlying tables.

Dimension conventions (staging layer)

- Apply consistent business naming to `label:` for every non-hidden dimension.
- Every dimension created or modified in staging carries a `description`.
- Use `group_label:` to group related dimensions in the field picker.
- Order labels by prefixing with space characters (e.g. `" my field label"`) to sort to the top.
- Apply consistent formatting to date, currency, numeric fields.
- Hide fields not relevant to business users — PKs, FKs, internal flags.

Measure conventions (aggregate layer)

- Every measure carries a `description` explaining its calculation methodology and business logic.
- Every measure has a `value_format_name` (e.g. `gbp`, `usd`, `decimal_2`, `percent_1`) — consistent formatting is non-negotiable.
- Use `sql_distinct_key` where required for accurate `count_distinct` aggregations.
- Provide `drill_fields:` for common exploration paths.
- Where a metric is meaningful at multiple grains (day, week, month), define tiered versions rather than expecting users to figure it out.
- Group measures logically using `group_label:`.

Explore conventions (integration layer)

- Prefer curated, business-process-aligned Explores over one-Explore-per-table. A handful of well-named Explores (e.g. `sales_performance`, `customer_retention`) beats fifty database-shaped Explores nobody can navigate.
- Every Explore has a `label:` and a `description:`.
- Joins are ideally many-to-one for performance, using warehouse FK→PK relationships (`<entity>_fk` → `<entity>_pk`).
- No direct SQL or business logic in the integration layer.** No joins between columns that aren't declared relationships in the warehouse — if you need a join the warehouse doesn't support, that's a sign the warehouse needs a model added, not that you should patch it in LookML.
- Use Quick Starts and Aggregate Awareness to improve UX and query performance.

The four pillars of Looker testing

- Test as you go.** Every measure and dimension added is immediately tested in an Explore.
- LookML Validator** — Looker's built-in syntax check.
- Looker Assertions** — formal `test` objects for critical business logic.
- Content Validator** — always before raising a PR. It tells you which existing Looks & Dashboards your change will break. Fix or coordinate with the content owner before merging.

7.2 LOOKER PR CHECKLIST

Every Looker PR description includes this, ticked:

- ☐ My PR represents one logical piece of work (one story).
- ☐ I have used the appropriate refinement layer (staging for dimensions, aggregate for measures).
- ☐ Any view I have refined is refined only once in staging and once in aggregate.
- ☐ New aggregate measures use the `_calculated` suffix.
- ☐ I have added labels, group_labels and descriptions to all new objects.
- ☐ New measures have a `value_format_name`.
- ☐ I have reconciled new measures against a reliable source.
- ☐ I have run the Content Validator and assessed impact.
- ☐ I have run the LookML Validator and it passes.
- ☐ I have individually tested every new measure and dimension.
- ☐ My Looker instance has no residual unused models from development.

Full reference: *RA Development & Design Guide for dbt & Looker*. The LookML reference documentation follows the Diataxis style — keep contributions consistent.

7.3 TABLEAU → LOOKER MIGRATION

Where a migration involves replacing Tableau, follow the *Guide to Migrating from Tableau to Looker*. Headlines:

- Inventory every Tableau workbook, dashboard and calculated field before starting.
- Translate Tableau calculated fields into LookML measures in `aggregate/` — never recreate them as ad-hoc Looker custom fields.
- Map Tableau filters to Looker filter parameters at the Explore level.
- Validate row counts and totals between Tableau and Looker for every published dashboard before sunseting the Tableau equivalent.
- Don't sunset Tableau until end-user training (a separate `enablement` Wire project type) is delivered.

7.4 DAGSTER

Dagster is our orchestration tool for non-dbt-Cloud projects.

- **Asset-based, not job-based.** Define dbt models, BI refreshes and external tasks as Dagster assets. Use `dagster-dbt` for automatic dbt asset generation.
- One repository per client engagement, with `assets/`, `resources/`, `schedules/`, `sensors/`, `partitions/` as top-level subfolders.
- Asset keys use the same `<object>_dim` / `<object>_fact` naming as dbt models so downstream traceability is preserved.
- **Deployment:** Dagster Cloud where the client pays for it; otherwise a Cloud Run deployment from a private RA image. The Wire Framework's `deployment:generate` produces the runbook.
- Canonical internal example is the Harvest Forecast MCP server (Cloud Run), which doubles as the reference implementation.

7.5 GOOGLE BIGQUERY

- **Project naming:** `<client>-<env>` (e.g. `acmecorp-prod`, `acmecorp-dev`). Dev environments isolated per consultant via personal datasets (`<initials>-analytics`).
- **Region:** match the client's data-residency requirement. Default `europa-west2` (London) for UK clients, `us-central1` for US. **Never mix regions within a project** — cross-region queries fail or cost dramatically more.
- **Partitioning:** every fact table is partitioned on a `_ts` or `_dt` column. Default daily granularity unless the table is small (<1M rows) or queried only at month grain.
- **Clustering:** cluster on the highest-cardinality join keys used in filters (typically the primary key and the most-used foreign key).
- **Cost control:** every consultant has BigQuery usage visibility for their dev project; budget alerts set at 50%, 80%, 100%. Anything pushing past 50% goes to your Head of Delivery first.
- **Service accounts:** never use a personal account for production jobs. Service-account JSON keys are stored in GCP Secret Manager, not committed to git.

Further reading: *BigQuery Physical Design Considerations* and *BigQuery Billing Model* in Confluence.

7.6 DATA PLATFORM DEVELOPMENT (CROSS-CUTTING)

A few principles that apply across the whole stack:

- **The conceptual model gates everything.** Pipeline design, data-model spec, and dashboard mockups all depend on an approved conceptual entity model. The Wire Framework enforces this; manual development should respect the same order.
- **Pipeline design includes a Data Flow Diagram.** Mermaid `flowchart` showing source → ingestion → staging → warehouse → BI. Generated by `/dp:pipeline_design:generate`.
- **The data model artifact includes a Physical ERD.** Mermaid `erDiagram` with every warehouse model, all columns with types, PKs, FKs and relationship lines. The most important review gate in a full-platform engagement.
- **Cost should be in the pipeline design.** Replication scenarios should be sized and costed (MAR, query slot usage) so the client can make an informed choice. Where multiple replication strategies are plausible (raw CDC vs server-side view vs hybrid), present them as Scenario A / B / C with cost and freshness implications, and ask the client to choose.

ACROSS THE STACK

Whichever tool you're using — dbt, Looker, Fivetran, Dagster, BigQuery — the same discipline applies: design first, document the choice, test before merging, and keep business logic in the warehouse. The technology is a means to an end; consistency across engagements is what makes us trustworthy.

SECTION 8 Testing & QA

8.1 DBT — MINIMUM TESTS REQUIRED

Every model must have:

- 1 Entry in a `schema.yml` in the same directory as the model.
- 2 **Primary key** with `unique` and `not_null` tests.
- 3 **Foreign keys** with `relationships` tests pointing to the parent model's PK.
- 4 Integration models with multiple sources must have `dbt_utils.unique_combination_of_columns` covering the natural-key-plus-source combination.

Recommended additional tests:

- `accepted_values` on enum / status columns.
- `dbt_utils.not_null_where` for conditional requirements.
- `dbt_utils.expression_is_true` for cross-field validations (`end_date >= start_date`).
- Timestamp sanity checks (`created_ts <= current_timestamp()`).

8.2 DBT — TEST COVERAGE BY LAYER

LAYER	DOCS	PK	OTHER
Staging	100%	unique + not_null	Basic validation
Integration	As needed	unique + not_null (or unique_combination)	relationships for FKs
Warehouse	100%	unique + not_null	Comprehensive: relationships, business-logic validation

8.3 DBT — TEST SEVERITY

- Primary-key tests: always `error`.
- Foreign-key tests: usually `error`.
- Optional / nice-to-have validations: `warn`.

Default is `error`; only mark a test `warn` if you have a concrete reason and document it.

8.4 DBT — UNIT TESTS

Unit tests validate transformation logic by mocking model inputs and asserting expected outputs, without hitting the database. They are distinct from schema tests and data tests.

RA convention:

- **Required** for all warehouse-layer models with business logic (case statements, window functions, complex joins).
- **Recommended** for staging models with non-trivial transformations.
- Detailed guidance in the `dbt-unit-testing` skill — Model-Inputs-Outputs pattern, format selection, BigQuery caveats, production deployment.

8.5 DBT — TEST EXECUTION FLOW

Standard sequence (the Wire Framework `/dp:utils:run-dbt` does this automatically):

```
dbt run --select staging.*
dbt test --select staging.*
dbt run --select integration.*
dbt test --select integration.*
dbt run --select warehouse.*
dbt test --select warehouse.*
dbt test --select test_type:data
```

If tests fail at any stage, the transformation stops — errors are prevented from propagating to the warehouse layer.

8.6 LOOKER TESTING

See Section 7.2: the four pillars (test as you go, LookML Validator, Looker Assertions, Content Validator) and the PR checklist.

8.7 UAT

- UAT plan generated by `/dp:uat:generate`, mapped to functional requirements.
- Conducted with end users, not just the technical stakeholder.
- Outcomes recorded in the framework; do not proceed to deployment without `/dp:uat:review` returning approved.
- For dashboard-extension engagements, UAT is mandatory before merging to the production Looker branch.

8.8 CONTINUOUS QUALITY

- Daily dbt jobs run the full test suite.
- Critical models have hourly freshness monitoring.
- Test failures trigger Slack alerts to the engagement channel, severity matching test severity.
- For data-quality-sensitive engagements (financial, safeguarding), add Elementary or equivalent for trend-based anomaly detection — testing is binary pass/fail; monitoring is adaptive.

SECTION 9 Project planning & engagement models

Agile / time-and-materials is the majority of our work and is the default engagement model for any new piece of business unless there's a specific reason to use something else. It maps best to the exploratory, collaborative nature of BI and data-platform projects — requirements emerge as users see early outputs, source data surfaces surprises that change the design, and the value of a dashboard is best judged by the people using it once they have it in front of them.

Fixed-price / fixed-deliverables is the exception, used primarily for Google PSF (Partner Services Fund) engagements where the funding model requires it, and for the occasional short, tightly scoped piece of work where the deliverable genuinely is well understood at SOW signature.

9.1 AGILE / TIME-AND-MATERIALS (DEFAULT)

- Sprint cadence is fortnightly by default. Some engagements run weekly cycles where the work is fast-moving.
- Backlog management in Jira; the engagement lead grooms the backlog before each sprint planning session with the client.
- No fixed deliverables; a sprint goal and a velocity target instead. Direction can shift sprint-by-sprint based on what the previous sprint surfaced.
- Burndown and time-spent reported to the client weekly.
- Wire Framework is still used per-deliverable — a sprint typically contains one `dbt_development` or `dashboard_extension` project plus standalone tickets.
- Time is tracked daily in Harvest; hours billed are the engagement's revenue, so accuracy is critical.

9.2 FIXED-PRICE (EXCEPTION — TYPICALLY PSF)

- Scope locked at SOW signature. Any change requires a documented change request through your Head of Delivery before work starts.
- Wire `requirements` artifact must map every SOW deliverable (D1, D2, ...) to the framework artifacts that will produce it. This mapping is the contract between scope and execution.
- Estimation produces a fixed number of hours; budget monitored weekly via Harvest. At 50% spend you should be ~50% through scope; at 80% spend with significantly less than 80% complete, escalate.
- Out-of-scope requests get acknowledged but not actioned without a change order. "Yes, we can do that — we'll quote it separately" is the right answer, not silent expansion.
- Acceptance is per-deliverable, gated by UAT for end-user deliverables and internal QA for technical ones.
- **PSF-specific:** Google PSF engagements have additional submission and reporting requirements. Use the `psf-sow-validator` skill before submitting any PSF SOW.

9.3 HOW IT AFFECTS DELIVERY

ASPECT	AGILE / T&M	FIXED-PRICE
Scope changes	Backlog re-prioritisation	Change request first
Risk on slippage	Client bears it	RA bears it
Pressure for shortcuts	Lower	Higher — guard against
Docs overhead	Same	Same
Wire discipline	Same	Same
Margin protection	Hours billed = margin	Hours-vs-scope critical

PITFALL

The temptation in fixed-price is to skip framework steps (validation, review gates) to recover hours. Don't. A skipped validate step costs more in rework than the time it saves. If you're falling behind, raise it — there are legitimate options (re-scoping, change order, agreeing reduced delivery) and skipping QA is not one of them.

9.4 ESTIMATION

We estimate in hours (not story points) because we bill in hours. Rough rule of thumb for a Wire full-platform engagement:

PHASE	HOURS
Requirements	4–8
Design (conceptual + pipeline + data model + mockups)	16–24
Development (pipeline + dbt + semantic layer + dashboards)	40–80
Testing (data quality + UAT)	8–16
Deployment	4–8
Enablement (training + documentation)	8–16

Starting points, not commitments. Always size against the actual SOW.

SECTION 10 Releases, branches & PRs

10.1 GIT BRANCHING

Every piece of work happens on a feature branch — never on `main` or `master`. The Wire Framework `/dp:new` enforces this; if you're working manually, do the same:

```
git checkout -b feature/<jira-ticket-id>-short-description
# e.g. feature/RAP-118-orders-warehouse-models
```

Branch naming: `feature/`, `fix/` or `refinement/` prefix; Jira ticket ID; short kebab-case description.

10.2 COMMIT MESSAGES

- Imperative present tense: "add user_dim model", not "added" or "adds".
- First line under 72 characters.
- Reference the Jira ticket:
`[RAP-118] add orders warehouse fact and dimension models`.
- Long description after a blank line if the change needs context.

10.3 PULL REQUESTS

Every PR description includes:

- What** changed (one logical piece of work — one story).
- Why** — the Jira ticket link, and a sentence of context.
- How tested** — what dbt tests run, what Looker validators run, manual checks performed.
- Risk** — known impacts on existing Looks/Dashboards (Content Validator output for Looker; downstream model count for dbt).
- For Looker PRs, the PR checklist in Section 7.2.
- For dbt PRs, the pre-commit checklist in Section 10.4.

10.4 DBT PRE-COMMIT CHECKLIST

- Filename follows naming convention (Section 5.2)
- File in correct directory (Section 5.1)
- All refs and sources in `s_` CTEs at the top of the file
- Final CTE exists and is selected from
- 4-space indentation, lines under 80 chars
- All fields lowercase
- PK `<object>_pk` generated with `dbt_utils.surrogate_key()`
- FKs `<object>_fk` generated with `dbt_utils.surrogate_key()`
- Timestamps: `<event>_ts`
- Booleans: `is_` or `has_` prefix
- Explicit join types (`inner join`, `left join`, etc.)
- Field ordering correct (keys → attrs → metrics → bools → temporal)
- Configuration appropriate for the layer
- `schema.yml` entry exists
- PK has `unique` + `not_null` tests
- FKs have `relationships` tests
- Model and columns documented (staging or warehouse)
- No SQL reserved words as column names
- Singular object names throughout
- `dbt build` runs cleanly locally
- `sqlfluff lint` passes (if `.sqlfluff` present)

10.5 REVIEW PROCESS

- Internal peer review first.** A client picking up an error breaks trust; an internal peer picking it up improves the process.
- Reviewer checks PR description, runs the code locally where practical, verifies tests pass, and confirms the relevant checklist is genuinely complete (ticked boxes are no substitute for actual checking).
- After internal approval, PR moves to the client review branch if the engagement has one, or merges to `main` for solo / RA-internal projects.
- Stable releases** are tagged after the PR is merged and any production deployment has been smoke-tested.

10.6 RELEASE PROCESS (LOOKER EXAMPLE)

- PR raised from `feature/...` to the `qa` branch.
- After internal approval, release notes sent to business stakeholders for UAT on QA.
- On stakeholder approval, PR raised from `qa` to `production` (stable) branch.
- Production deployment runbook (`/dp:deployment:generate`) followed.
- Smoke-test in production. Monitor for 24 hours.

SECTION 11 Ways of working

11.1 WORKING AS A TEAM

- **Default to writing it down.** Decisions made in a meeting that aren't captured in Jira, Confluence or `status.md` didn't happen.
- **Default to async.** A Slack message with the right context beats a half-hour meeting. Meetings are for things that genuinely need real-time discussion: design reviews, retros, hard conversations.
- **Default to public.** Conversations about a client engagement happen in the client's Slack channel, not in DMs. The exception is anything HR-sensitive or commercially sensitive.
- **Default to challenging.** If you think a design decision is wrong, say so — explicitly, with reasoning. We hire people for judgement, not deference. Push back politely, hear the response, then commit to whatever the team lands on.
- **Default to documenting the deviation.** If you can't follow a standard for a good reason on a particular project, document why in `status.md` (Wire) or the project `README.md` (manual). Silent deviations are the ones that cause confusion later.

11.2 COMMUNICATION CHANNELS

CHANNEL	USED FOR
Slack — client channel	All day-to-day project communication
Slack — #delivery	Cross-team delivery, blockers, escalations
Slack — #general	Company-wide news
Slack DM	Personal / private / HR-sensitive
Email	Externally to client stakeholders only — internal email is rare
Jira	All tickets, sprint planning, stand-up updates
Confluence	Long-form docs, SOPs, design decisions, RFC-style proposals
Fathom	Every client meeting recorded with permission
Harvest	Time tracking — submitted daily

11.3 STAND-UPS & MEETINGS

- Daily stand-ups on engagements with 2+ delivery consultants. 15 min max. What I did, what I'm doing, blockers.
- Weekly delivery review with Head of Delivery — engagement health, hours vs scope, risks, escalations.
- Fortnightly retros at end of each sprint on retainer engagements.
- Monthly all-hands.
- Quarterly business review.

11.4 FATHOM USAGE

- Record every client meeting with stated permission. Permission is recorded in the Fathom share settings.
- Recordings auto-publish to the client engagement Slack channel.
- Action items captured by Fathom's AI are reviewed and turned into Jira tickets by the engagement lead within 24 hours.
- Sensitive conversations (HR, commercial negotiation, internal disputes) are not recorded.

11.5 HARVEST USAGE

- Time tracked daily against the correct project and task code.
- Where a client doesn't use Harvest project codes, use the RA internal equivalent.
- Non-billable time (internal training, PD, admin) tracked too — it informs utilisation reporting and Forecast accuracy.
- The Harvest MCP server is available for ad-hoc queries.

11.6 GOOGLE WORKSPACE USAGE

- **Sheets** for data manipulation that doesn't merit a dbt model — quick analyses, client data, finance models. Anything reproducible goes into dbt instead.
- **Slides** for client-facing presentations. Use the RA template; Wire has a Google Slides skill for generating decks from project artifacts.
- **Docs** for working drafts and collaboration with clients who prefer it over Confluence. Final versions migrated to Confluence at engagement end.
- **Drive** for file storage; mirrors engagement structure (`/Clients/<Client>/<Engagement>/`).

11.7 CLAUDE USAGE

- Claude Code is the runtime for the Wire Framework. Always use it from inside a configured project repo.
- Claude.ai (web/desktop) is fine for ad-hoc analytical work, drafting, document review, exploratory thinking.
- **Sensitive client data does not go into a personal Claude account.** Use the RA Team plan, where retention and confidentiality terms are configured.
- Where Claude produces client-facing content, the consultant is responsible for reviewing it before it ships. Don't copy-paste blind.

SECTION 12 Jira

12.1 PROJECT STRUCTURE

One Jira project per client engagement (long-running clients) or per delivery workstream (shorter engagements). Project keys are short and memorable — typically a 3–6 letter abbreviation derived from the client name or workstream.

12.2 HIERARCHY

- **Epic** — a deliverable (e.g. "D1 — Live Pastoral Data Pipeline").
- **Task** — a unit of work within a deliverable, typically 2–16 hours.
- **Sub-task** — a sub-unit of a task, used sparingly.

Wire `/dp:utils:jira-create` creates this hierarchy automatically from the `requirements` artifact: one epic per SOW deliverable, tasks per Wire artifact. Use on every full-platform engagement.

12.3 WORKFLOW STATES

Standard: `To Do → In Progress → In Review → Done`. Engagements with formal QA add `In QA` between Review and Done.

12.4 HYGIENE

- Move tickets through states as you work — don't batch updates at end of day.
- Add a comment when handing off ("Passing to <name> for review — see PR #123").
- Tickets must be linked to a parent epic or a sprint (or both).
- Estimates added at sprint planning, not retrospectively.
- `/dp:utils:jira-sync` keeps Wire artifact status in line with Jira state — run after each `/dp:*.review` step.

SECTION 13 Confluence

13.1 SPACE STRUCTURE

SPACE	PURPOSE
RA	Internal RA methodology, SOPs, framework docs
DEL	Wire Framework documentation, consultant handbook
PD	Internal platform / tooling documentation
BIZ	Business operations, career framework, policy
<Client>	One space per client engagement

13.2–13.3 PAGE CONVENTIONS & WHAT GOES WHERE

- Page titles are descriptive, not cute. "BigQuery cost optimisation for the platform migration", not "BQ cost notes".
- Every page has a clear owner in the page metadata.
- Pages older than 12 months without an update get reviewed at the quarterly business review — confirmed current, updated, or marked deprecated.
- Deprecated pages get `[deprecated]` appended to the title and a link at the top to the replacement.
- Long-form docs / SOPs / design decisions / post-mortems → Confluence. Working drafts with the client → Google Docs, migrated at engagement end. Real-time discussion → Slack. Wire artifacts → in-repo markdown under `.agent_v2/`; final approved versions summarised onto Confluence at engagement end.

13.4 ATlassian MCP

Both Confluence and Jira are accessible via the Atlassian MCP server inside Claude Code. Use it for cross-referencing during Wire artifact generation (`/dp:utils:atlassian-search` finds existing client documentation).

SECTION 14 GitHub

14.1 REPOSITORY STRUCTURE

- One repo per client engagement, named `<client>-data-platform` or `<client>-workstream`.
- Wire projects nest under a top-level `dbt/`, `looker/`, `pipeline/`, `dagster/` structure as appropriate.
- `README.md` at repo root describes engagement, stack, and how to get started locally.
- `CONTRIBUTING.md` at repo root references this standards document.

14.2 ACCESS

- All RA staff are members of the `rittmananalytics` GitHub org.
- Contractors get added per-repo, not org-wide.
- 2FA is mandatory — without it you can't commit to any RA repo.

14.3 BRANCH PROTECTION

- `main` is protected — direct pushes blocked, PRs require at least one review, status checks (CI tests, sqlfluff) must pass.
- Looker `production` (or `stable`) branches are similarly protected.

14.4 CI/CD

- dbt projects use GitHub Actions for CI: on every PR, run `dbt build --select state:modified+` (Slim CI) against a CI dataset, plus `sqlfluff lint`.
- Looker projects use Looker's built-in deployment via the Git repo connection; LookML Validator runs in Looker, not in GitHub Actions.
- Pipeline code (Python, Dagster) is tested in GitHub Actions with `pytest` against mocked sources.

14.5 SECRETS

- Never commit credentials. Never. Pre-commit hooks check for common patterns; `gitLeaks` runs in CI.
- Secrets go in GCP Secret Manager (or client equivalent), retrieved at runtime by the service account.
- Local development uses `gcloud auth application-default login`, never service-account keys.

SECTION 15 Harvest & Harvest Forecast

15.1 TIME TRACKING (HARVEST)

- Track time daily.
- Use the correct project and task code; if unsure, ask the engagement lead before guessing.
- Comments on time entries are encouraged for any non-trivial block of work — they help with future estimation.
- Submit weekly timesheets by end of Friday.

15.2 RESOURCE SCHEDULING (FORECAST)

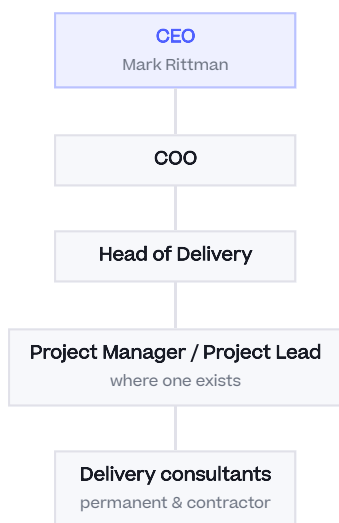
- Head of Delivery owns the Forecast schedule.
- Consultants check their Forecast weekly and flag conflicts immediately.
- Time off and non-billable allocations (training, PD) go into Forecast so utilisation reports stay accurate.

15.3 RA HARVEST FORECAST MCP

We run an internal MCP server (deployed to Google Cloud Run) that exposes Harvest Forecast as MCP tools. Connect it in your Claude Code config — it lets the Wire Framework query availability when scoping engagements.

SECTION 16 Line management & escalation

16.1 REPORTING STRUCTURE



- PM is a dedicated role on larger engagements; on smaller ones a **Project Lead** (typically a Senior Analytics Consultant also doing delivery) fills the role informally. Both function the same way for escalation.
- Not every engagement has a PM or Project Lead.** Short or solo engagements may run with a single consultant reporting directly to Head of Delivery. The chain collapses where the role doesn't exist; it does not invent a layer that isn't there.
- CRO and other non-delivery functions report into the CEO directly.

16.3 CONTRACTOR SPECIFICS

- Contracts signed before the first day of work, not after.
- Day rate and invoicing terms agreed in writing before starting.
- Contractors are part of the team for the engagement — included in stand-ups, retros, Slack, all client communication. Treated identically to permanent staff on delivery matters.
- All delivery output (code, documents, dashboards) is the client's or RA's property per the engagement contract; contractors don't retain rights to engagement output.
- Notice periods agreed in the contract — typically two weeks either way for long engagements, none for short ones.

16.2 WHAT TO ESCALATE, AND TO WHOM

Shortest-route-up: try the next person up the chain, and only skip a level where that level is unavailable, conflicted, or explicitly out of scope (e.g. safeguarding).

MATTER	ESCALATE TO
Day-to-day delivery questions	PM / Project Lead first; → Head of Delivery if unavailable.
Technical blockers >1 day	PM / Project Lead first to coordinate response; → Head of Delivery if unresolved after another day.
Hours/scope variance >10% (fixed-price)	PM / Project Lead immediately , they raise to Head of Delivery same day. If none, → Head of Delivery directly, before continuing.
Client relationship issues	PM / Project Lead within 24 hours; they judge whether to involve Head of Delivery. If the issue concerns the PM themselves, → Head of Delivery directly.
Commercial questions	PM / Project Lead → Head of Delivery → COO/CEO as needed.
HR matters	Head of Delivery directly, regardless of whether a PM exists. If the matter concerns the Head of Delivery, → COO.
Safeguarding / safety / compliance	CEO directly, no intermediaries. Anything that touches a child, anything illegal, anything that risks data breach.

DON'T SIT ON THESE

Client relationship signals — frustration, escalating tone, signs of dissatisfaction — escalate within 24 hours. They get worse, not better, if left.

SECTION 17 Final notes & maintenance

This document is a living standard. It will be updated when:

- The Wire Framework adds a new project type, command or convention.
- A client engagement surfaces a pattern worth encoding (a new SQL dialect translation, a new testing convention, a new tool we've adopted).
- We hire a new role with different needs (designers, business analysts, etc.).

Updates go through PR review like any other deliverable. **Owner:** Head of Delivery. **Cadence:** review every six months at minimum, or when any of the trigger events above occurs.

IF YOU SPOT A CONTRADICTION

Raise it. Either the engagement is deviating from the standard (which needs to be documented and justified) or the standard is out of date (which needs to be updated). Both outcomes are good.

A Appendix · Reference links

TOPIC	SOURCE
Wire Framework operating manual	<i>Wire Framework Consultant Handbook</i> · Confluence DEL · 3300065281
dbt & Looker design philosophy	<i>RA Development & Design Guide for dbt & Looker</i> · Confluence PD · 3001024586
Cross-engagement SOP index	<i>RA Standard Operating Procedure Directory</i> · Confluence RA · 2897870895
Wire Framework white paper	<i>LLM-Orchestrated End-to-End Data Platform Delivery</i> · Confluence DEL · 3300130817
Career progression	<i>Career Progression Framework (2024)</i> · Confluence BIZ · 2522054664
Estimation reference	<i>Review of our units of measurement for estimating</i> · Confluence RA Delivery
BigQuery physical design	<i>BigQuery Physical Design Considerations</i> · Confluence
BigQuery billing model	<i>BigQuery Billing Model</i> · Confluence
Tableau → Looker migration	<i>Guide to Migrating from Tableau to Looker</i> · Confluence

End of document · v1.0 · 12 May 2026 · Owner: Head of Delivery